

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

ассистент

должность, уч. степень, звание

подпись, дата

Ю. В. Ветрова

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 9

ИСПОЛЬЗОВАНИЕ БИБЛИОТЕК JAVASCRIPT ДЛЯ АНИМАЦИИ
ЭЛЕМЕНТОВ И ВИЗУАЛИЗАЦИИ ДАННЫХ

по курсу:

WEB-ТЕХНОЛОГИИ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4326

подпись, дата

Г. С. Томчук

инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Цель работы: с использованием библиотек языка JavaScript научиться создавать анимационные эффекты и строить графики на веб-странице.

2 Задание

Задание состоит из двух пунктов. В процессе выполнения можно использовать любые библиотеки языка JavaScript, включая те, работа с которыми была рассмотрена в лекционном курсе (jQuery и Chart.js).

1. Создать анимационный эффект на веб-странице, установив такие параметры анимации, как длительность, функцию плавности (ее часто называют функцией сглаживания, или кривой анимации, представляющей собой график изменения параметра анимации в зависимости от времени) и др.

Рекомендация: желательно разработать анимационный эффект (один или несколько) с учетом тематики курсовой работы, чтобы иметь возможность применения созданной анимации на практике.

2. Поместить на веб-страницу график (диаграмму), отображающий данные в соответствии с тематикой, соответствующей варианту задания:

Вариант 6. Самые крупные операторы сотовой связи в России (по количеству абонентов).

Примечание. Разрешается отобразить на графике данные по своему выбору, не входящие ни в один из предложенных вариантов. Вид графика (диаграммы) должен соответствовать характеру отображаемых данных. Количество точек на графике (элементов диаграммы) должно быть примерно от 5 до 7.

3 Ход выполнения

В ходе выполнения лабораторной работы была создана веб-страница, демонстрирующая использование JavaScript-библиотек для анимации элементов интерфейса и визуализации данных. Для реализации анимационного эффекта была выбрана библиотека anime.js, а для построения диаграммы использовалась библиотека Chart.js.

Сначала была подготовлена структура проекта. В папке лабораторной работы были созданы файлы index.html, style.css и script.js. Также были добавлены локальные копии библиотек anime.js и Chart.js, чтобы страница могла корректно работать без постоянного подключения к внешним CDN-серверам. На рисунке 1 изображена структура файлов проекта.



Рисунок 1 — Файлы веб-страницы

В файле index.html была создана основная разметка страницы. В нее вошли заголовочный блок лабораторной работы, область с анимированной SIM-картой, информационные карточки с параметрами анимации, область диаграммы и таблица исходных данных. Также в HTML-файле были подключены CSS-стили, библиотеки JavaScript и основной пользовательский скрипт. На рисунке 2 изображен фрагмент HTML-разметки страницы.

```
36 <section class="stats-grid" aria-label="Ключевые данные">
37 <article class="stat-card">
38 <span>Длительность</span>
39 <strong>900 мс</strong>
40 <p>Появление карточек с небольшой последовательной задержкой.</p>
41 </article>
42 <article class="stat-card">
43 <span>Easing</span>
44 <strong>easeOutElastic</strong>
45 <p>Элементы движутся мягко и немного пружинят в финале.</p>
46 </article>
47 <article class="stat-card">
48 <span>График</span>
49 <strong>Chart.js</strong>
50 <p>Горизонтальная диаграмма сравнивает число абонентов.</p>
51 </article>
52 </section>
53
54 <section class="chart-section">
55 <div class="section-heading">
56 <p class="eyebrow">Вариант 6</p>
57 <h2>Самые крупные операторы сотовой связи в России</h2>
58 <p>
59 Количество абонентов приведено в миллионах. Для федеральных операторов
60 использованы данные CNews Analytics на 31.12.2023; Yota добавлена как
61 крупнейший российский MVNO по данным TelecomDaily/MForum за 2025 год.
62 </p>
63 </div>
64
65 <div class="chart-wrap">
66 <canvas
67 id="operatorsChart"
68 aria-label="Диаграмма операторов связи по количеству абонентов"
69 ></canvas>
70 </div>
```

Рисунок 2 — Разметка анимированных карточек и секции с графиком

Затем был оформлен внешний вид страницы в файле style.css. Для страницы были заданы цветовая схема, отступы, адаптивная сетка, стили карточек, кнопки, области диаграммы и декоративной SIM-карты. Отдельное внимание было уделено адаптивности: при уменьшении ширины экрана основные блоки перестраиваются в одну колонку, что позволяет удобно просматривать страницу на мобильных устройствах. Также были использованы CSS-переменные, сохраненные в элемент «:root». На рисунке 3 изображен фрагмент CSS-стилей страницы.

```
1 :root {
2   --bg: #f6f7fb;
3   --text: #172033;
4   --muted: #5d667a;
5   --panel: #ffffff;
6   --line: #dde3ef;
7   --accent: #0a8f7a;
8   --accent-2: #e84a5f;
9   --accent-3: #2f6fed;
10  --accent-4: #f2a516;
11  --shadow: 0 18px 50px rgba(22, 30, 48, 0.12);
12 }
13
14 * {
15   box-sizing: border-box;
16 }
17
18 body {
19   margin: 0;
20   background: var(--bg);
21   min-height: 100vh;
22   color: var(--text);
23   font-family: Arial, Helvetica, sans-serif;
24 }
25
26 .page {
27   margin: 0 auto;
28   padding: 32px 0 24px;
29   width: min(1180px, calc(100% - 32px));
30 }
31
32 .intro {
33   display: grid;
34   grid-template-columns: minmax(0, 1.25fr) minmax(280px, 0.75fr);
35   align-items: stretch;
36   gap: 28px;
37   min-height: 390px;
38 }
```

Рисунок 3 — Фрагмент CSS-стилей страницы, сверху указаны CSS-переменные

После этого была реализована анимация элементов с помощью библиотеки anime.js. Для текстовых элементов был настроен эффект плавного появления со смещением по вертикали. Для информационных карточек использовалась последовательная анимация с задержкой между элементами, изменением прозрачности, масштаба и положения. Для SIM-карты была добавлена бесконечная анимация покачивания, а для кругов сигнала — циклическое изменение масштаба и прозрачности. В параметрах анимации были явно заданы длительность, задержка, функция плавности и режим

повторения. На рисунке 4 изображен фрагмент JavaScript-кода с настройкой анимации.

```
8
9 function runIntroAnimation() {
10   anime({
11     targets: ".intro__content > *",
12     translateY: [28, 0],
13     opacity: [0, 1],
14     delay: anime.stagger(130),
15     duration: 900,
16     easing: "easeOutCubic",
17   });
18
19   anime({
20     targets: ".stat-card",
21     translateY: [34, 0],
22     scale: [0.94, 1],
23     opacity: [0, 1],
24     delay: anime.stagger(140, { start: 350 }),
25     duration: 900,
26     easing: "easeOutElastic(1, .72)",
27   });
28
29   anime({
30     targets: "#simCard",
31     translateY: [-10, 10],
32     rotate: [-2, 2],
33     duration: 2400,
34     direction: "alternate",
35     loop: true,
36     easing: "easeInOutSine",
37   });
38
39   anime({
40     targets: ".signal",
41     scale: [0.75, 1.1],
42     opacity: [0.75, 0.12],
43     delay: anime.stagger(420),
44     duration: 1800,
45     direction: "alternate",
46     loop: true,
47     easing: "easeInOutQuad",
48   });
49 }
50
```

Рисунок 4 — JS-код с определением анимаций

Далее была реализована диаграмма с использованием библиотеки Chart.js. В качестве темы был выбран вариант 6: самые крупные операторы сотовой связи в России по количеству абонентов. Для отображения данных была выбрана горизонтальная столбчатая диаграмма, так как она хорошо подходит для сравнения нескольких категорий между собой. На диаграмме представлены пять операторов: МТС, МегаФон, Ростелеком / Т2, Билайн и Yota. Значения указаны в миллионах абонентов. На рисунке 5 изображены фрагменты JavaScript-кода, отвечающего за построение диаграммы.

```

1  const operators = [
2    { name: "МТС", subscribers: 81.8 },
3    { name: "МегаФон", subscribers: 67.7 },
4    { name: "Ростелеком / Т2", subscribers: 48.1 },
5    { name: "Билайн", subscribers: 44.1 },
6    { name: "Yota", subscribers: 9.5 },
7  ];
8
51 function createOperatorsChart() {
52   const canvas = document.getElementById("operatorsChart");
53
54   return new Chart(canvas, {
55     type: "bar",
56     data: {
57       labels: operators.map((operator) => operator.name),
58       datasets: [
59         {
60           label: "Количество абонентов, млн",
61           data: operators.map((operator) => operator.subscribers),
62           backgroundColor: ["#0a8f7a", "#2f6fed", "#e84a5f", "#f2a516", "#6a5acd"],
63           borderRadius: 6,
64           borderSkipped: false,
65         },
66       ],
67     },
68     options: {
69       indexAxis: "y",
70       responsive: true,
71       maintainAspectRatio: false,
72       animation: {
73         duration: 1200,
74         easing: "easeOutQuart",
75       },
76       plugins: {
77         legend: {
78           display: false,
79         },
80         tooltip: {
81           callbacks: {
82             label(context) {
83               return `${context.parsed.x} млн абонентов`;
84             },
85           },
86         },
87         title: {
88           display: true,
89           text: "Крупнейшие операторы по количеству абонентов",
90           color: "#172033",
91           font: {

```

Рисунок 5 — Определение массива данных и создание горизонтальной столбчатой диаграммы по этим данным

В результате страница отображает анимированный блок, информационные карточки, таблицу данных и диаграмму с количеством абонентов операторов сотовой связи. На рисунке 6 изображен итоговый вид веб-страницы в браузере.

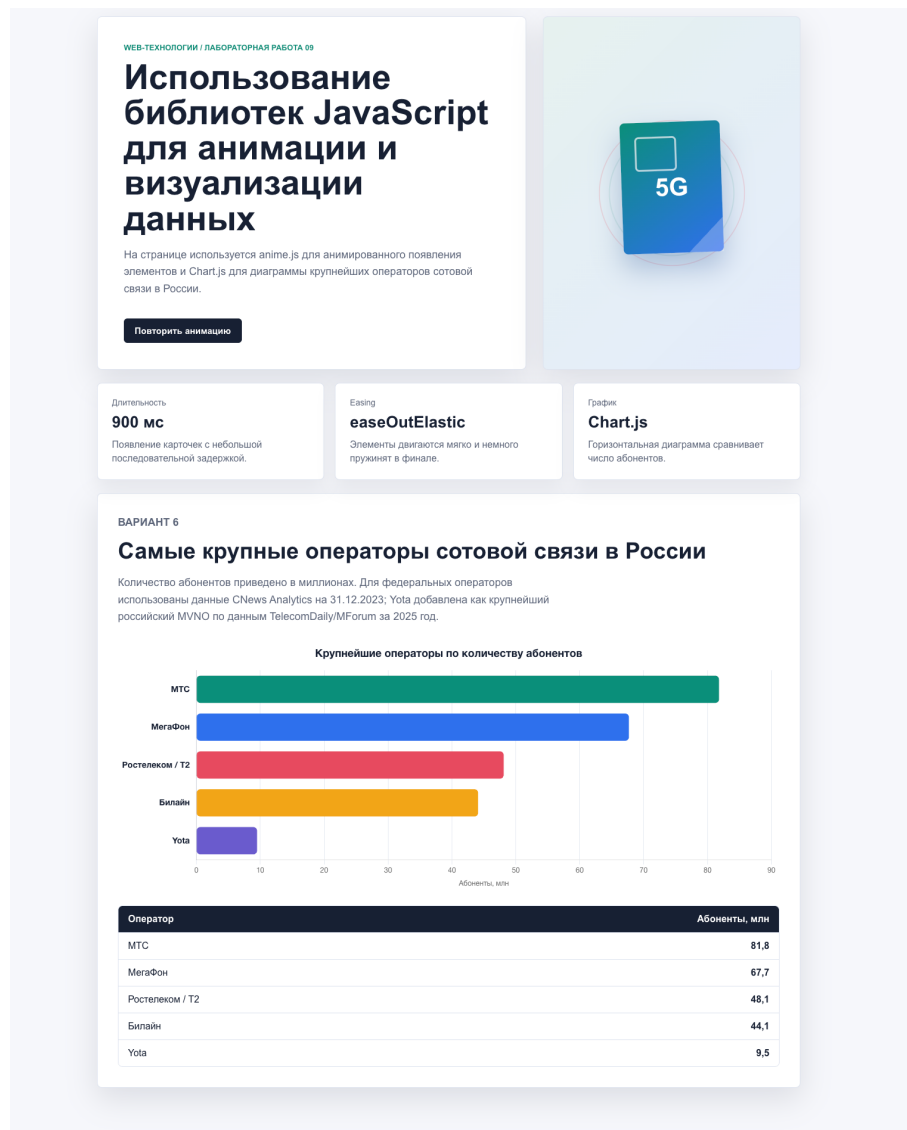


Рисунок 6 — Итоговая страница

4 Выводы

В ходе лабораторной работы были изучены и применены возможности JavaScript-библиотек для создания анимации и визуализации данных на веб-странице. С помощью библиотеки anime.js был реализован анимационный эффект с заданными параметрами длительности, задержки, функции плавности и повторения. С помощью библиотеки Chart.js была построена горизонтальная столбчатая диаграмма, наглядно отображающая количество абонентов крупнейших операторов сотовой связи в России.

В результате выполнения работы была создана полноценная веб-страница, объединяющая интерактивную анимацию, стилизованный интерфейс и графическое представление данных. Работа показала, что

использование готовых JavaScript-библиотек позволяет значительно упростить создание динамичных и информативных веб-интерфейсов.

ПРИЛОЖЕНИЕ

Листинг 1 — index.html

```
<!doctype html>
<html lang="ru">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <link rel="stylesheet" href="style.css" />
    <script src="vendor/anime.min.js" defer></script>
    <script src="vendor/chart.umd.min.js" defer></script>
    <script src="script.js" defer></script>
  </head>
  <body>
    <main class="page">
      <section class="intro">
        <div class="intro__content">
          <p class="eyebrow">Web-технологии / Лабораторная работа 09</p>
          <h1>Использование библиотек JavaScript для анимации и визуализации
и данных</h1>
          <p class="lead">
            На странице используется anime.js для анимированного появления элементов
и
            Chart.js для диаграммы крупнейших операторов сотовой связи в России.
          </p>
          <button class="replay-btn" type="button" id="replayAnimation">
            Повторить анимацию
          </button>
        </div>

        <div class="phone-scene" aria-label="Анимированная SIM-карта">
          <div class="signal signal--one"></div>
          <div class="signal signal--two"></div>
          <div class="sim-card" id="simCard">
            <span></span>
            <strong>5G</strong>
          </div>
        </div>
      </section>

      <section class="stats-grid" aria-label="Ключевые данные">
        <article class="stat-card">
          <span>Длительность</span>
          <strong>900 мс</strong>
          <p>Появление карточек с небольшой последовательной задержкой.</p>
        </article>
        <article class="stat-card">
          <span>Easing</span>
          <strong>easeOutElastic</strong>
          <p>Элементы двигаются мягко и немного пружинят в финале.</p>
        </article>
        <article class="stat-card">
          <span>График</span>
          <strong>Chart.js</strong>
          <p>Горизонтальная диаграмма сравнивает число абонентов.</p>
        </article>
      </section>

      <section class="chart-section">
        <div class="section-heading">
```

```

    <p class="eyebrow">Вариант 6</p>
    <h2>Самые крупные операторы сотовой связи в России</h2>
    <p>
        Количество абонентов приведено в миллионах. Для федеральных операторов
        использованы данные CNews Analytics на 31.12.2023; Yota добавлена как
        крупнейший российский MVNO по данным TelecomDaily/MForum за 2025 год.
    </p>
</div>

<div class="chart-wrap">
    <canvas
        id="operatorsChart"
        aria-label="Диаграмма операторов связи по количеству абонентов"
    ></canvas>
</div>

<div class="data-table" aria-label="Таблица данных для диаграммы">
    <div class="data-row data-row--head">
        <span>Оператор</span>
        <span>Абоненты, млн</span>
    </div>
    <div class="data-row"><span>МТС</span><span>81,8</span></div>
    <div class="data-row"><span>МегаФон</span><span>67,7</span></div>
    <div class="data-row"><span>Ростелеком / Т2</span><span>48,1</span></div>
    <div class="data-row"><span>Билайн</span><span>44,1</span></div>
    <div class="data-row"><span>Yota</span><span>9,5</span></div>
</div>
</section>
</main>
</body>
</html>

```

Листинг 2 — script.js

```

const operators = [
    { name: "МТС", subscribers: 81.8 },
    { name: "МегаФон", subscribers: 67.7 },
    { name: "Ростелеком / Т2", subscribers: 48.1 },
    { name: "Билайн", subscribers: 44.1 },
    { name: "Yota", subscribers: 9.5 },
];

function runIntroAnimation() {
    anime({
        targets: ".intro__content > *",
        translateY: [28, 0],
        opacity: [0, 1],
        delay: anime.stagger(130),
        duration: 900,
        easing: "easeOutCubic",
    });

    anime({
        targets: ".stat-card",
        translateY: [34, 0],
        scale: [0.94, 1],
        opacity: [0, 1],
        delay: anime.stagger(140, { start: 350 }),
        duration: 900,
        easing: "easeOutElastic(1, .72)",
    });
}

```

```

anime({
  targets: "#simCard",
  translateY: [-10, 10],
  rotate: [-2, 2],
  duration: 2400,
  direction: "alternate",
  loop: true,
  easing: "easeInOutSine",
});

anime({
  targets: ".signal",
  scale: [0.75, 1.1],
  opacity: [0.75, 0.12],
  delay: anime.stagger(420),
  duration: 1800,
  direction: "alternate",
  loop: true,
  easing: "easeInOutQuad",
});
}

function createOperatorsChart() {
  const canvas = document.getElementById("operatorsChart");

  return new Chart(canvas, {
    type: "bar",
    data: {
      labels: operators.map((operator) => operator.name),
      datasets: [
        {
          label: "Количество абонентов, млн",
          data: operators.map((operator) => operator.subscribers),
          backgroundColor: ["#0a8f7a", "#2f6fed", "#e84a5f", "#f2a516", "#6a5acd"],
          borderRadius: 6,
          borderSkipped: false,
        },
      ],
    },
    options: {
      indexAxis: "y",
      responsive: true,
      maintainAspectRatio: false,
      animation: {
        duration: 1200,
        easing: "easeOutQuart",
      },
      plugins: {
        legend: {
          display: false,
        },
        tooltip: {
          callbacks: {
            label(context) {
              return `${context.parsed.x} млн абонентов`;
            },
          },
        },
      },
      title: {
        display: true,
        text: "Крупнейшие операторы по количеству абонентов",
        color: "#172033",
      },
    },
  });
}

```

```

        font: {
            size: 18,
            weight: "bold",
        },
        padding: {
            bottom: 18,
        },
    },
},
scales: {
    x: {
        beginAtZero: true,
        title: {
            display: true,
            text: "Абоненты, млн",
        },
        grid: {
            color: "#e7ebf3",
        },
    },
    y: {
        grid: {
            display: false,
        },
        ticks: {
            color: "#172033",
            font: {
                size: 14,
                weight: "bold",
            },
        },
    },
},
},
},
});
}

window.addEventListener("DOMContentLoaded", () => {
    runIntroAnimation();
    createOperatorsChart();

    document.getElementById("replayAnimation").addEventListener("click", () => {
        anime.remove(".intro__content > *, .stat-card");
        runIntroAnimation();
    });
});

```

Листинг 3 — styles.css

```

:root {
    --bg: #f6f7fb;
    --text: #172033;
    --muted: #5d667a;
    --panel: #ffffff;
    --line: #dde3ef;
    --accent: #0a8f7a;
    --accent-2: #e84a5f;
    --accent-3: #2f6fed;
    --accent-4: #f2a516;
    --shadow: 0 18px 50px rgba(22, 30, 48, 0.12);
}

```

```

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  background: var(--bg);
  min-height: 100vh;
  color: var(--text);
  font-family: Arial, Helvetica, sans-serif;
}

.page {
  margin: 0 auto;
  padding: 32px 0 24px;
  width: min(1180px, calc(100% - 32px));
}

.intro {
  display: grid;
  grid-template-columns: minmax(0, 1.25fr) minmax(280px, 0.75fr);
  align-items: stretch;
  gap: 28px;
  min-height: 390px;
}

.intro__content,
.phone-scene,
.chart-section {
  box-shadow: var(--shadow);
  border: 1px solid var(--line);
  border-radius: 8px;
  background: var(--panel);
}

.intro__content {
  display: flex;
  flex-direction: column;
  justify-content: center;
  padding: 44px;
}

.eyebrow {
  margin: 0 0 12px;
  color: var(--accent);
  font-weight: 700;
  font-size: 13px;
  letter-spacing: 0;
  text-transform: uppercase;
}

h1,
h2 {
  margin: 0;
  letter-spacing: 0;
}

h1 {
  max-width: 760px;
  font-size: clamp(32px, 5vw, 58px);
  line-height: 1.02;
}

```

```

h2 {
  font-size: clamp(25px, 3vw, 38px);
  line-height: 1.12;
}

.lead,
.section-heading p {
  max-width: 760px;
  color: var(--muted);
  font-size: 18px;
  line-height: 1.58;
}

.replay-btn {
  transition:
    transform 180ms ease,
    background 180ms ease;
  cursor: pointer;
  margin-top: 18px;
  border: 0;
  border-radius: 6px;
  background: var(--text);
  padding: 12px 18px;
  width: fit-content;
  color: #fff;
  font-weight: 700;
  font-size: 15px;
}

.replay-btn:hover {
  transform: translateY(-2px);
  background: #263148;
}

.phone-scene {
  display: grid;
  position: relative;
  place-items: center;
  background: linear-gradient(135deg, rgba(10, 143, 122, 0.12), rgba(47, 111, 237, 0.13)), #fff;
  min-height: 320px;
  overflow: hidden;
}

.sim-card {
  display: grid;
  position: relative;
  place-items: center;
  box-shadow: 0 24px 48px rgba(33, 81, 144, 0.32);
  border-radius: 8px;
  background: linear-gradient(145deg, #0a8f7a, #2f6fed);
  width: 168px;
  height: 220px;
  color: #fff;
}

.sim-card::before {
  position: absolute;
  top: 24px;
  left: 24px;
  border: 3px solid rgba(255, 255, 255, 0.66);

```

```

border-radius: 7px;
width: 64px;
height: 52px;
content: "";
}

.sim-card span {
position: absolute;
right: 0;
bottom: 0;
clip-path: polygon(100% 0, 0 100%, 100% 100%);
background: rgba(255, 255, 255, 0.25);
width: 58px;
height: 58px;
}

.sim-card strong {
font-size: 42px;
}

.signal {
position: absolute;
border: 2px solid rgba(10, 143, 122, 0.38);
border-radius: 50%;
}

.signal--one {
width: 210px;
height: 210px;
}

.signal--two {
border-color: rgba(232, 74, 95, 0.32);
width: 285px;
height: 285px;
}

.stats-grid {
display: grid;
grid-template-columns: repeat(3, minmax(0, 1fr));
gap: 18px;
margin: 22px 0;
}

.stat-card {
box-shadow: 0 12px 30px rgba(22, 30, 48, 0.08);
border: 1px solid var(--line);
border-radius: 8px;
background: var(--panel);
padding: 24px;
min-height: 152px;
}

.stat-card span {
display: block;
margin-bottom: 10px;
color: var(--muted);
font-size: 14px;
}

.stat-card strong {
display: block;

```

```

    font-size: 27px;
}

.stat-card p {
    margin: 10px 0 0;
    color: var(--muted);
    line-height: 1.45;
}

.chart-section {
    padding: 34px;
}

.section-heading {
    margin-bottom: 24px;
}

.chart-wrap {
    position: relative;
    padding: 12px 4px 0;
    min-height: 420px;
}

.data-table {
    margin-top: 26px;
    border: 1px solid var(--line);
    border-radius: 8px;
    overflow: hidden;
}

.data-row {
    display: grid;
    grid-template-columns: 1fr 160px;
    gap: 16px;
    border-top: 1px solid var(--line);
    background: #fff;
    padding: 13px 16px;
}

.data-row:first-child {
    border-top: 0;
}

.data-row--head {
    background: var(--text);
    color: #fff;
    font-weight: 700;
}

.data-row span:last-child {
    font-weight: 700;
    text-align: right;
}

.footer {
    padding: 20px 4px 0;
    color: var(--muted);
    font-size: 14px;
}

@media (max-width: 840px) {
    .page {

```



```

padding-top: 16px;
width: min(100% - 24px, 680px);
}

.intro,
.stats-grid {
  grid-template-columns: 1fr;
}

.intro__content,
.chart-section {
  padding: 24px;
}

.phone-scene {
  min-height: 270px;
}

.chart-wrap {
  min-height: 360px;
}

.data-row {
  grid-template-columns: 1fr 110px;
}
}

```